

MiniRISC Assignment 1A: ISA Planning and Processor Architecture

Computer Organization and Architecture Laboratory · IIT Kharagpur · Submission for the 5 October 2026 deadline

Group members: [add names and roll numbers]

1. Processor architecture

Figure 1 is the block-level architecture of the planned 32-bit MiniRISC processor. It separates the datapath (solid blocks and buses) from the control path (the control unit and the accent-coloured signals inside each block), and marks the future Booth multiplier / MAC as a placeholder attached to the ALU.

One control unit steers the datapath; the multiplier plugs into the ALU

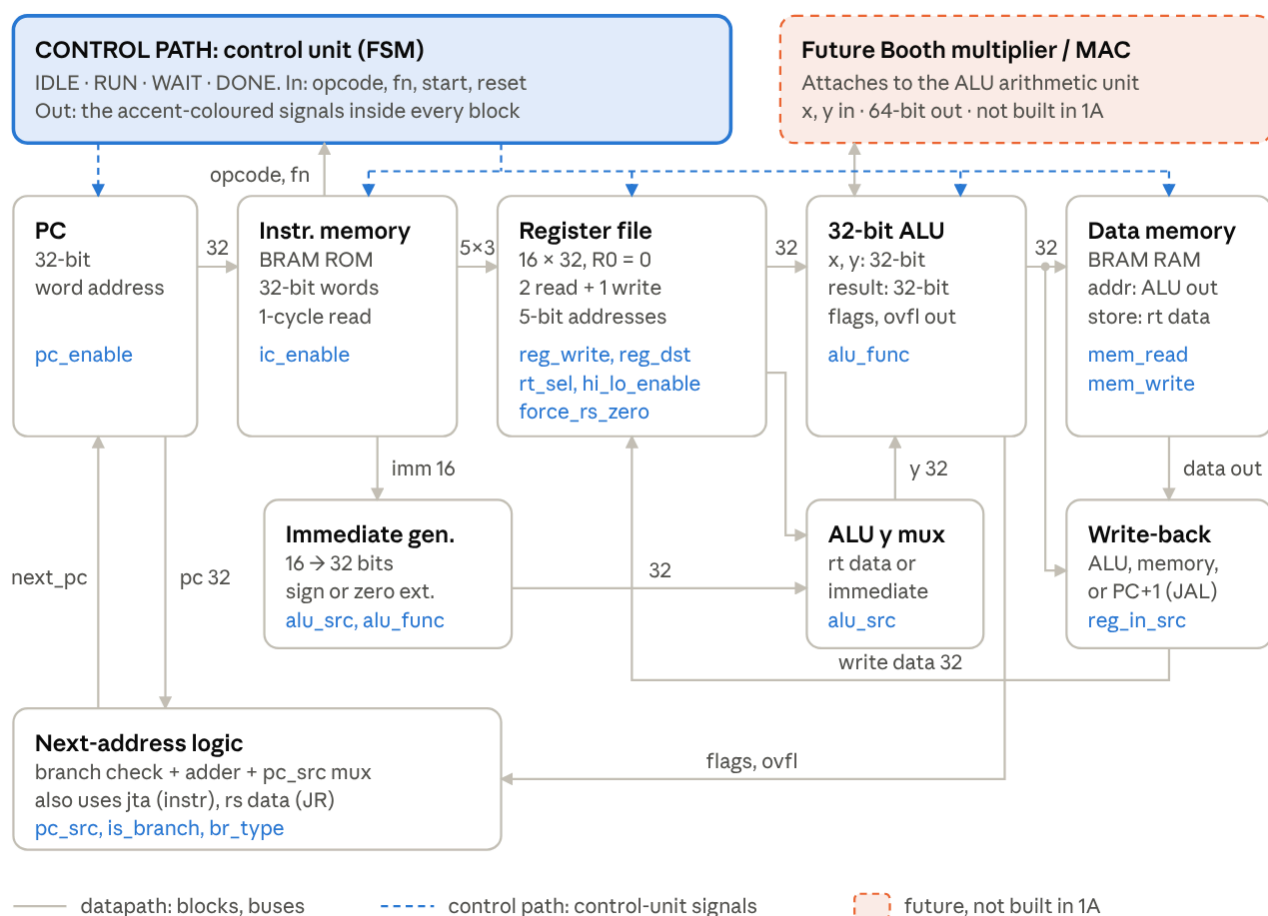


Figure 1 · MiniRISC block diagram: datapath, control path and future multiplier placeholder

Figure 1. MiniRISC block diagram. Solid boxes and grey lines are the datapath; the control unit and the dashed accent lines and text are the control path; the dashed block is the future multiplier / MAC.

Notes on Figure 1

- **Instruction memory** is a single-port BRAM ROM holding one 32-bit instruction per word address. It is addressed with `next_pc`, so the PC and the fetched instruction update on the same clock edge, which absorbs the one-cycle BRAM read latency.
- **Data memory** is a single-port BRAM RAM. Address = ALU result, store data = the register-file `rt` port. Its read is synchronous, so `LD` takes two cycles; the control FSM holds the PC in a WAIT state.
- **ALU input multiplexers.** The ALU's x input is always the `rs` read port. The y input is chosen by the ALU y mux (register data or immediate). In front of the register file, `force_rs_zero` (2:1) and `rt_sel` (5 used inputs) multiplex the read addresses.
- **Next-address logic** checks the branch condition from the ALU flags, adds the offset (gated by the branch result) to PC + 1 with a single adder, and selects the next PC with `pc_src`: PC + 1 (+ offset), jump target `jta`, register `rs` (for `JR`), or 0 (start-up).
- **Write-back mux** chooses ALU result, memory data or PC + 1 (the `JAL` link) for the register file's single write port.
- **Control unit** is a 4-state FSM (IDLE, RUN, WAIT, DONE). It reads only `opcode` and `fn`, plus `start` and `reset`.

Checklist: required diagram elements and where they appear in Figure 1

#	Required element	In Figure 1
1	PC and PC-update path	PC block; Next-address logic; <code>pc</code> and <code>next_pc</code> buses
2	Instruction input / instruction-memory interface	Instr. memory (BRAM ROM), 32-bit instruction, <code>ic_enable</code>
3	Instruction decoder / control unit	Control unit (FSM), fed by <code>opcode</code> , <code>fn</code>
4	16×32 register file, two read ports, one write port	Register file block
5	Immediate-generation logic	Immediate gen. (16 → 32, sign or zero extend)
6	ALU input multiplexers	ALU y mux; <code>force_rs_zero</code> and <code>rt_sel</code> address muxes
7	32-bit ALU	32-bit ALU

#	Required element	In Figure 1
8	Branch-decision / PC-selection path	Next-address logic with ALU flags, <code>pc_src</code> , <code>is_branch</code> , <code>br_type</code>
9	Data-memory interface	Data memory (BRAM RAM), <code>mem_read</code> , <code>mem_write</code>
10	Write-back path	Write-back mux and the write-data bus to the register file
11	Future multiplier / MAC integration point	Dashed block above the ALU
12	Major buses and widths	Labelled on the buses
13	Major control signals	Accent-coloured text in every block

Basic processor requirements

Requirement	How it is met
Sixteen 32-bit registers R0–R15	16×32 register array; R15 doubles as the link register for <code>JAL</code>
Two read ports, one write port	Yes; the single write port is fed by the write-back mux
R0 hard-wired to zero	R0 always reads 0 and writes to it are ignored
32-bit PC	Yes; counts words ($PC \leftarrow PC + 1$)
32-bit ALU	Load-upper, compare/set, arithmetic, logic and shift units
Decode and control logic	Instruction fields wired directly; control-unit FSM
Immediate generation	16 → 32 bits, sign or zero extension
Load/store, branch/program control	<code>LD</code> , <code>ST</code> ; compare-and-branch, <code>J</code> , <code>JAL</code> , <code>JR</code> , <code>NOP</code> , <code>HALT</code>
Provision for the Booth multiplier	<code>MUL</code> / <code>MULU</code> slot in the ALU arithmetic unit, HI/LO registers, hardware stall
Instruction and data memory interfaces	Single-port BRAM ROM and BRAM RAM

HI and LO are two extra 32-bit special registers kept beside the 16×32 array. They are written only by `MULU` and read only through `MFHI` / `MFL0`, so the programmer-visible general-purpose file is exactly 16×32 with `R0 = 0`.

2. Complete planned instruction list

Group	Mnemonic	Syntax	Operation
Control	NOP	<code>NOP</code>	$PC \leftarrow PC + 1$
Control	HALT	<code>HALT</code>	Stop; PC frozen until reset
Arithmetic	ADD, SUB	<code>ADD rd, rs, rt</code>	$rd \leftarrow rs + rt, rd \leftarrow rs - rt$
Arithmetic	ADDI, SUBI	<code>ADDI rd, rs, imm</code>	$rd \leftarrow rs \pm \text{sext}(\text{imm})$
Arithmetic	MUL	<code>MUL rd, rs, rt</code>	$rd \leftarrow \text{low 32 bits of } rs \times rt$ (signed)
Arithmetic	MULU	<code>MULU rs, rt</code>	$\{HI, LO\} \leftarrow rs \times rt$ (64-bit, unsigned)
Logic	AND, OR, NOR, XOR	<code>AND rd, rs, rt</code>	bitwise on rs and rt
Logic	NOT	<code>NOT rd, rs</code>	$rd \leftarrow \sim rs$
Logic immediate	ANDI, ORI, NORI, XORI	<code>ORI rd, rs, imm</code>	$rd \leftarrow rs \text{ op } \text{zext}(\text{imm})$
Shift	SLL, SRL, SRA	<code>SLL rd, rs, rt</code>	shift rs left / logical right / arithmetic right by rt
Shift immediate	SLLI, SRLI, SRAI	<code>SLLI rd, rs, imm</code>	shift rs by imm
Set on compare	SLT, SGT, SLE, SGE, SEQ, SNE	<code>SLT rd, rs, rt</code>	$rd \leftarrow 1$ if the relation holds, else 0
Set immediate	SLTI, SGTI, SLEI, SGEI, SEQI, SNEI	<code>SLTI rd, rs, imm</code>	same, against sext(imm)
Constants	LI	<code>LI rd, imm</code>	$rd \leftarrow \text{sext}(\text{imm})$
Constants	LUI	<code>LUI rd, imm</code>	$rd \leftarrow \text{imm} \ll 16$

Group	Mnemonic	Syntax	Operation
Register transfer	MOVE	MOVE rd, rs	$rd \leftarrow rs$
HI/LO read	MFHI, MFLO	MFHI rd	$rd \leftarrow HI, rd \leftarrow LO$
Memory	LD	LD rd, imm(rs)	$rd \leftarrow MEM[rs + sext(imm)]$
Memory	ST	ST rd, imm(rs)	$MEM[rs + sext(imm)] \leftarrow rd$
Jump	J	J target	$PC \leftarrow \text{target}$ (26-bit word address)
Jump	JAL	JAL target	$R15 \leftarrow PC + 1; PC \leftarrow \text{target}$
Jump	JR	JR rs	$PC \leftarrow rs$
Branch	BEQ, BNE	BEQ rs, rd, label	if $rs = rd$ ($\neq$): $PC \leftarrow PC + 1 + sext(\text{offset})$
Branch	BLT, BLE, BGT, BGE	BLT rs, rd, label	same, on $rs < rd, \leq, >, \geq$
Branch	BZ	BZ rs, label	branch if $rs = 0$
Branch	BV	BV rs, rd, label	branch if $rs - rd$ overflows

Total: 53 instructions (each opcode / function counted once).

Coverage of the required instruction classes

Required class	MiniRISC instructions
Arithmetic (ADD, SUB, ADDI, SUBI)	ADD, SUB, ADDI, SUBI; INC and DEC are ADDI rd, rd, 1 and SUBI rd, rd, 1
Logical (AND, OR, XOR, NOR, NOT)	All five, plus ANDI, ORI, XORI, NORI
Shift (SLL, SRL, SRA; register or immediate amount)	SLL, SRL, SRA (register amount); SLLI, SRLI, SRAI (immediate amount)
Comparison (SLT, SGT)	SLT, SGT, plus SLE, SGE, SEQ, SNE and immediate forms
Memory (LD, ST, base + offset)	LD, ST with imm(rs) addressing

Required class	MiniRISC instructions
Branch and control	Unconditional J, JR ; conditional BZ and six compare-and-branch forms; NOP ; HALT . BMI is BLT rs, R0, label ; BPL is BGE rs, R0, label
Register transfer and constants	MOVE, LI, LUI (LUI + ORI builds any 32-bit constant)
Multiplication	MUL, MULU, with MFHI / MFLO to read the 64-bit result

Additional instructions and their justification

Instruction(s)	Justification
SLE, SGE, SEQ, SNE and their immediate forms	Complete set of six relations from one compare unit, so loop conditions need no extra inversion step
BEQ, BNE, BLT, BLE, BGT, BGE	Compare two registers and branch in one instruction; no flag register; BZ, BMI and BPL are special cases
BV	Branches on signed overflow of rs – rd; useful for checking results in later multiply / accumulate programs
JAL, JR	Subroutine call and return for the later matrix and image programs
MULU, MFHI, MFLO	Make the full 64-bit product visible, and give a place to accumulate later

3. Instruction formats

All instructions are 32 bits. Fields have the same position in every format, and **the destination register is always bits 25:21.**

Format	31:26	25:21	20:16	15:11	10:6	5:0
R	op	rd	rs	rt	shamt (reserved)	fn

Format	31:26	25:21	20:16	15:11	10:6	5:0
I (ALU immediate, LI, LUI, LD, ST)	op	rd	rs	imm[15:0] (overlaps rt / shamt / fn)		
Branch	op	rs2	rs	offset[15:0]		
J	op	jta[25:0]				

- **Source and destination fields.** Destination `rd` is bits 25:21; sources `rs` (20:16) and `rt` (15:11). In the I format the second source is the immediate. For `ST` the `rd` field holds the register whose value is stored; in the branch format the first register field `rs2` is the second compared register and is not written.
- **Function field.** Only R-type instructions (`op` = 100000) use `fn` (bits 5:0) as the ALU function. In all other formats those bits belong to the immediate, so every I-type instruction has its own opcode.
- **Special fields for shifts.** The shift amount comes from the second ALU operand: register `rt` (R-type) or `imm` (I-type), using its low five bits. The `shamt` field is reserved and not used.
- **Special encodings for control.** `NOP` is opcode 000000 (an all-zero word); `HALT` is opcode 111111 (an all-ones opcode).

4. Opcode allocation

op range	Class	Opcodes
000000	No-op	NOP
010000–010111	Conditional branches	BEQ 010000, BZ 010001, BNE 010010, BLT 010011, BLE 010100, BGT 010101, BGE 010110, BV 010111
100000–100111	R-type, constants, moves, memory	R-type 100000, MFHI 100001, LI 100010, LUI 100011, MOVE 100100, LD 100101, ST 100110, MFLO 100111
101000–101011	Jumps	J 101000, JAL 101001, JR 101011 (101010 reserved)
110000–111110	I-type ALU	ADDI 110000, SUBI 110001, ANDI 110010, ORI 110011, NORI 110100, XORI 110101, SLLI 110110, SRLI 110111, SRAI 111000, SLTI 111001, SGTI 111010, SLEI 111011, SGEI 111100, SEQI 111101, SNEI 111110

op range	Class	Opcodes
111111	Stop	HALT

All unassigned opcodes are illegal and stop the processor (the FSM goes to DONE).

R-type function codes (fn). The upper three bits select the ALU unit and the lower three select the operation, so fn is the ALU control code itself and I-type instructions reuse the same codes.

fn[5:3]	ALU unit	fn[2:0] operations
000	Load-upper	LUI 000
001	Compare / set	SLT 000, SGT 001, SLE 010, SGE 011, SEQ 100, SNE 101
010	Arithmetic	ADD 000, SUB 001, MUL 010, MULU 011
011	Logic	AND 000, OR 001, NOT 010, NOR 011, XOR 100
100	Shift	SLL 000, SRL 001, SRA 010

Rationale. Opcode prefix 11 means ALU-with-immediate and 01 means conditional branch, so the class is visible from two bits. NOP is all zeros so cleared memory is harmless; HALT uses all ones. The I-type opcodes follow the R-type function order. MUL , MULU and NOT have no immediate form.

5. Immediate, branch and memory-addressing conventions

Immediates. A 16-bit field imm[15:0] , extended to 32 bits. It is sign-extended for ADDI , SUBI , LI , the set-on-compare immediates, and load/store and branch offsets. It is zero-extended for the logic immediates (ANDI ORI NORI XORI), so that LUI rd, hi16 followed by ORI rd, rd, lo16 builds any 32-bit constant. LUI places imm in bits 31:16 and clears the low half. Shift immediates use the low five bits.

Branch displacement. Branch format op | rs2 | rs | offset[15:0] . When taken, $PC \leftarrow PC + 1 + \text{sext}(\text{offset})$, with the offset counted in instructions (words), a reach of about $\pm 32K$ instructions. The conditions compare rs with the register in the rs2 field, or with zero for BZ . There is no condition-code register and no delay slot. J and JAL use a 26-bit absolute word address; JR jumps to the address in rs .

Memory addressing. Base register plus signed 16-bit offset: effective address = $rs + \text{sext}(\text{imm})$, formed by the ALU adder. `LD rd, imm(rs)` loads a 32-bit word into `rd`; `ST rd, imm(rs)` stores `rd`. Only word accesses exist, so no alignment logic is needed. Instruction and data memories are single-port BRAMs.

PC convention. The PC is word-addressed: sequential flow is $PC \leftarrow PC + 1$.

6. Planned MUL / MAC semantics

The ISA includes `MUL` and `MULU`. A MAC instruction is not planned in this assignment, but the design leaves room for it.

Question	MUL	MULU
Encoding	<code>MUL Rd, Rs1, Rs2</code> (R-type, fn = 010010)	<code>MULU Rs1, Rs2</code> (R-type, fn = 010011)
Source registers	rs, rt	rs, rt
Destination	rd	HI and LO (implicit)
Signed?	Yes: two's-complement operands	No: unsigned operands
Architecturally visible part of the 64-bit product	Low 32 bits, product[31:0]	All 64 bits: HI = product[63:32], LO = product[31:0]
Reading the result	directly in rd	<code>MFHI rd</code> , <code>MFL0 rd</code>
Multi-cycle?	Yes, expected (Booth multiplier)	Yes, expected
Stalling or NOPs	Hardware stall: the control FSM holds the PC and instruction in a WAIT state for the multiplier latency (a placeholder of 33 wait cycles); software NOPs are not needed, so programs do not depend on the latency	Same

Integration point. The multiplier attaches to the ALU arithmetic unit as the `MUL` / `MULU` function slot (marked in Figure 1). The upper half of the product goes to HI through the `hi_lo_enable` path, and the lower half goes to LO (or to `rd` for `MUL`).

MAC later. Because the product is kept in HI:LO, a MAC ($\text{HI:LO} \leftarrow \text{HI:LO} + \text{rs} \times \text{rt}$) can be added with only a new function code in the arithmetic group and an adder on the accumulate path; no format change is needed.

7. Justification of the ISA organisation

- **Load/store, three-operand, fixed 32-bit format.** Simple decode: every field is wired straight out of the instruction word, and memory is touched only by **LD** and **ST**.
- **Destination always in bits 25:21.** One write-address source for almost every instruction, and a 16-bit immediate fits in bits 15:0.
- **fn equals the ALU control code.** R-type needs no ALU decoder; I-type instructions get their own opcodes (the immediate removes **fn**) and reuse the same codes.
- **Compare-and-branch and set-on-compare, no flag register.** No hidden state between instructions; **BZ** / **BMI** / **BPL** are special cases of the same hardware.
- **Pseudo-instructions enforced in hardware.** **LI**, **MOVE**, **MFHI**, **MFLO** and **BZ** reuse **ADD** / **SUB** with an operand forced to **R0**, so they do not depend on the assembler zeroing unused fields.
- **HI/LO for the multiplier.** Keeps the 16×32 register file at two read ports and one write port, exposes the full 64-bit product, and gives a ready accumulator for a later MAC.
- **Word-addressed PC with PC + 1.** No alignment bits; one adder serves sequential flow, branch target and the **JAL** link.
- **Hardware stall for multi-cycle operations.** **LD** and the multiplier share one **WAIT** mechanism, and the programmer never inserts **NOPs**.